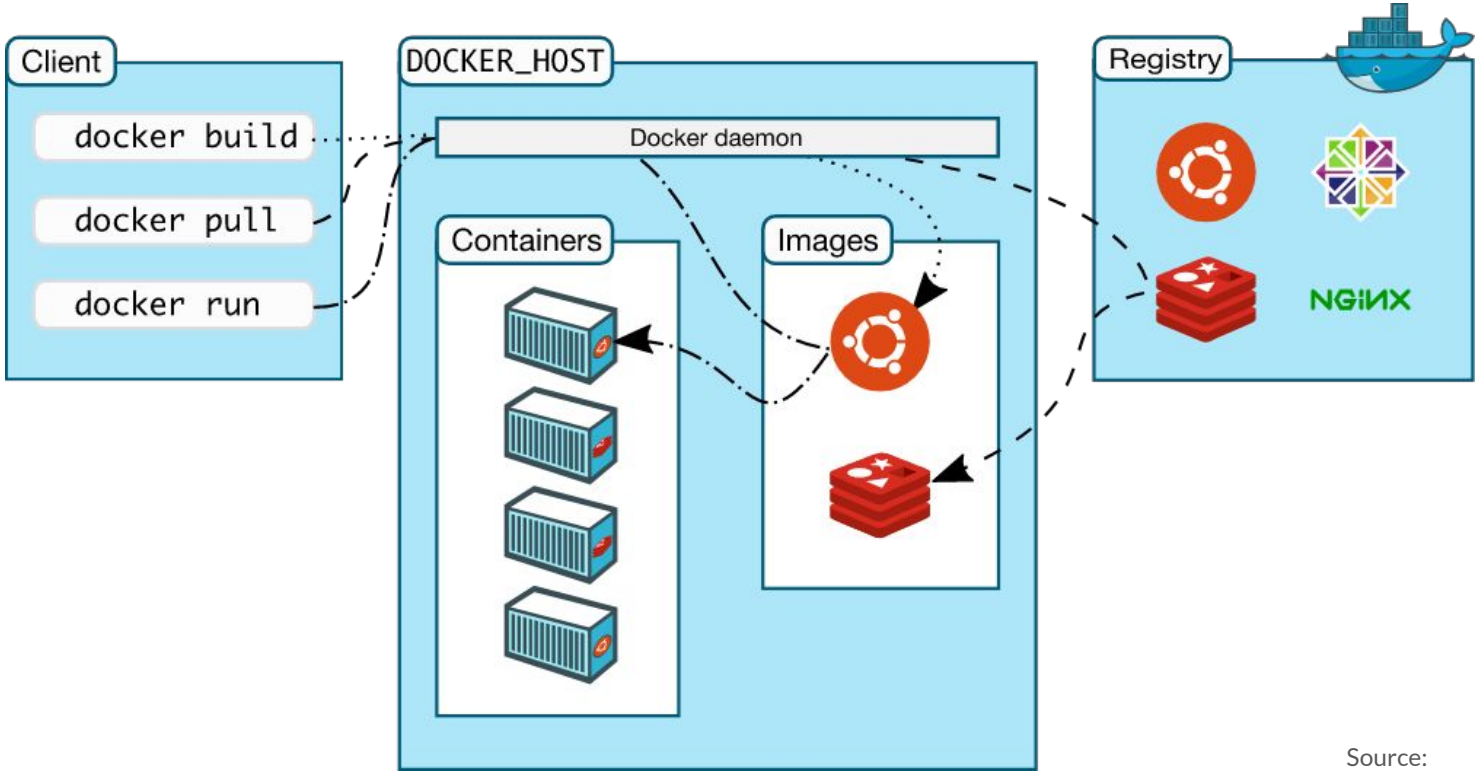
Introduction to Docker

Overview: What is Docker?

- Allows for easy deployment of software in a loosely isolated sandbox (containers).
- Can run many containers simultaneously on a given host.
- Containers are lightweight and contain everything needed to run the application, so you do not need to rely on what is currently installed on the host.
- The key benefit of Docker is that it allows users to package an application with **all of its dependencies into a standardized unit**.

Overview: Docker Architecture



Source:
https://docs.docker.com/get-started/overview/

Images

- Read-only template with instructions for creating a Docker container.
- Often, an image is based on another image, with some additional customization.
- You might create your own images or you might only use those created by others and published in a registry.

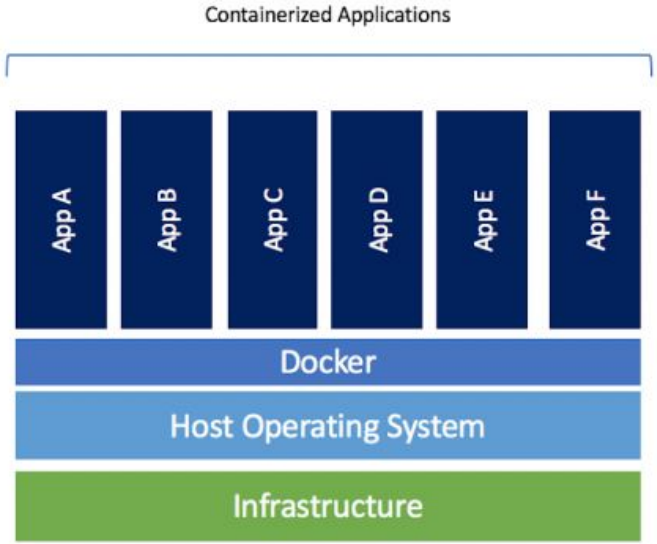
Images

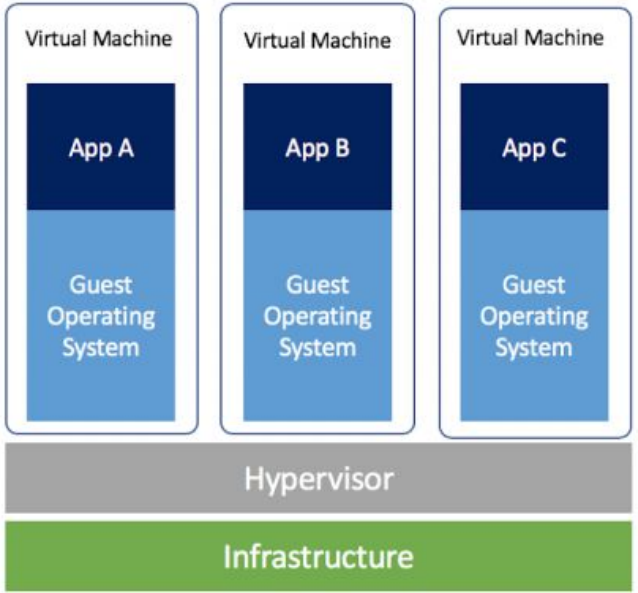
- To build your own image, you create a Dockerfile with a simple syntax for defining the steps needed to create the image and run it.
- Each instruction in a Dockerfile creates a layer in the image. When you change the Dockerfile and rebuild the image, only those layers which have changed are rebuilt.
- This is part of what makes images so lightweight, small, and fast, when compared to other virtualization technologies.

Containers

- Runnable instance of an image. You can create, start, stop, move, or delete a container using the Docker API or CLI. You can connect a container to one or more networks, attach storage to it.
- By default, a container is relatively well isolated from other containers and its host machine. You can control how isolated a container's network, storage, or other underlying subsystems are from other containers or from the host machine.
- A container is defined by its image as well as any configuration options you provide to it when you create or start it.
- When a container is removed, any changes to its state that are **not** stored in persistent storage disappear.

Overview: How is it Different from VMs?





Source: https://www.docker.com/blog/containers-replacing-virtual-machines/

Commonly Used Commands

```
$ docker pull [OPTIONS]
NAME[:TAG|@DIGEST]
```

Pull an image or a repository from a registry

https://docs.docker.com/engine/reference/commandline/pull/

Commonly Used Commands

`$ docker run [OPTIONS] IMAGE [COMMAND] [ARG…]`

First creates a writeable container layer over the specified image, and then starts it using the specified command.

Notable options:

- `--name`: Assign name to container
- `-it`: Combination of both --interactive(-i) and --tty(-t): keep STDIN open even if not attached, and allocate a pseudo-TTY
- `-v`: Bind mount a volume (more on this later)
- `--rm`: Automatically remove the container when it exits
- `--net/--network`: Connect a container to a network (more on this later)
- `-p`: Publish a container's port(s) to the host

https://docs.docker.com/engine/reference/commandline/run/

Commonly Used Commands

```
$ docker build [OPTIONS] PATH | URL | -
```

Build an image from a Dockerfile. A build's context is the set of files located in the specified `PATH` or `URL`.

Notable options:

- `-f`: Name of the Dockerfile, default is 'PATH/Dockerfile'
- `-force-rm`: Always remove intermediate containers
- `-no-cache`: Do not use cache when building the image

https://docs.docker.com/engine/reference/commandline/build/

Commonly Used Commands

```
$ docker ps [OPTIONS]
```

List containers

Notable options:

- `-a`: Show all containers, running and stopped
- `-s`: Display total file size

https://docs.docker.com/engine/reference/commandline/ps/

Commonly Used Commands

```
$ docker images [OPTIONS] [REPOSITORY[:TAG]]
```

List images

Notable options:

- `-a`: Show all images, default hides intermediate images

https://docs.docker.com/engine/reference/commandline/images/

Commonly Used Commands

```
$ docker rm [OPTIONS] CONTAINER [CONTAINER…]
```

Remove one or more containers

Notable options:

- `-f`: Force the removal of a running container (SIGKILL)

https://docs.docker.com/engine/reference/commandline/rm/

Commonly Used Commands

```
$ docker rmi [OPTIONS] IMAGE [IMAGE…]
```

Remove one or more images

Notable options:

- `-f`: Force the removal of a image

https://docs.docker.com/engine/reference/commandline/rmi/

Commonly Used Commands

```
$ docker exec [OPTIONS] CONTAINER COMMAND [ARG…]
```

Run a command in a running container. Usually used to open a interactive bash in a running container.

Notable options:

`-it`: combination of both --interactive(-i) and --tty(-t): keep STDIN open even if not attached, and allocate a pseudo-TTY

https://docs.docker.com/engine/reference/commandline/exec/

Commonly Used Commands

```
$ docker cp [OPTIONS] CONTAINER:SRC_PATH DEST_PATH
```

Or:

```
$ docker cp [OPTIONS] SRC_PATH CONTAINER:DEST_PATH
```

Copy files/folders between a container and the local filesystem. Behaves like the UNIX `cp`, and will have similar options for copying recursively, etc.

https://docs.docker.com/engine/reference/commandline/cp/

Dockerfile

Dockerfiles are instruction for Docker to build images automatically. Using `docker build` users can create an automated build that executes several command-line instructions in succession.

Dockerfile Syntax

In general, the format is:

```
# Comment

INSTRUCTION arguments
```

https://docs.docker.com/engine/reference/builder/

Dockerfile Syntax

Docker runs instructions in a `Dockerfile` in order. A `Dockerfile` **must begin with a** `FROM` **instruction**. The `FROM` instruction specifies the Parent Image from which you are building. `FROM` may only be preceded by one or more `ARG` instructions, which declare arguments that are used in `FROM` lines in the `Dockerfile`.

https://docs.docker.com/engine/reference/builder/

Dockerfile Syntax

The `RUN` instruction has 2 forms:

```
RUN <command>
```

shell form, the command is run in a shell. Default is `/bin/sh -c` on linux, or `cmd /S /C` on Windows

```
RUN ["executable", "param1", "param2"]
```

exec form

https://docs.docker.com/engine/reference/builder/

Dockerfile Syntax

The `CMD` instruction has 3 forms:

```
CMD [“executable”, “param1”, “param2”]
```

exec form, this is the preferred form

```
CMD [“param1”, “param2”]
```

As default parameters to `ENTRYPOINT`

```
CMD command param1 param2
```

shell form

https://docs.docker.com/engine/reference/builder/

Dockerfile Syntax

There can only be one `CMD` instruction in a `Dockerfile`. If you list more than one `CMD` then only the last will take effect.

The **main purpose** of `CMD` is to provide defaults for an executing container. These defaults can include an executable, or they can omit the executable, in which case you must specify an `ENTRYPOINT` instruction as well.

https://docs.docker.com/engine/reference/builder/

Dockerfile Syntax

`ENV <key>=<value> …`

The `ENV` instruction sets the environment variable `<key>` to the value `<value>`. This value will be interpreted for other environment variables.

The environment variables set using `ENV` **will persist** when a container is run from the resulting image.

https://docs.docker.com/engine/reference/builder/

Dockerfile Syntax

Alternatively you can use `ARG <key>=<value> …`

`ARG` **will not persist** when a container is run from the resulting image.

https://docs.docker.com/engine/reference/builder/

Dockerfile Syntax

The `COPY` instruction copies new files or directories from <src> and adds them to the filesystem of the container at the path <dest>.

The `COPY` instruction has 2 forms:

```
COPY [--chown=<user>:<group>] <src>... <dest>

COPY [--chown=<user>:<group>] [“<src>”,... “<dest>”]
```

The latter form is required for paths containing whitespace

https://docs.docker.com/engine/reference/builder/

Dockerfile Syntax

The `ENTRYPOINT` instruction has 2 forms:

```
ENTRYPOINT [“executable”, “param1”, “param2”]
```

exec form, this is the preferred form

```
ENTRYPOINT command param1 param2
```

shell form

An `ENTRYPOINT` allows you to configure a container that will run as executable.

https://docs.docker.com/engine/reference/builder/

Dockerfile Example

```
FROM osrf/ros:foxy-desktop

SHELL ["/bin/bash", "-c"]

# dependencies

RUN apt-get update --fix-missing && \

    apt-get install -y git \

                       nano \

                       vim \

                       python3-pip \

                       tmux
```

Parent image

Specify shell

`apt-get` dependencies, note the use of line escape for neater formatting

https://github.com/f1tenth/f1tenth_gym_ros/blob/ros2_port/Dockerfile

Dockerfile Example

```
RUN pip3 install --upgrade pip
RUN pip3 install numpy \
                 scipy \
                 Pillow \
                 gym \
                 pyyaml \
                 llvmlite \
                 numba \
                 pyglet \
                 transforms3d
```

`pip` dependencies, note the use of line escape for neater formatting

https://github.com/f1tenth/f1tenth_gym_ros/blob/ros2_port/Dockerfile

Dockerfile Example

```
# f1tenth gym
RUN git clone https://github.com/f1tenth/f1tenth_gym
RUN cd f1tenth_gym && \
    pip3 install -e gym/

# ros2 gym bridge
RUN mkdir -p sim_ws/src/f1tenth_gym_ros
COPY . /sim_ws/src/f1tenth_gym_ros
RUN source /opt/ros/foxy/setup.bash && \
    cd sim_ws/ && \
    rosdep install -i --from-path src --rosdistro foxy -y && \
    colcon build
```

`git` dependencies, the directory you go into after `cd` when called by `RUN` does **NOT** persist.

Copying the current directory into the container.

https://github.com/f1tenth/f1tenth_gym_ros/blob/ros2_port/Dockerfile

Dockerfile Example

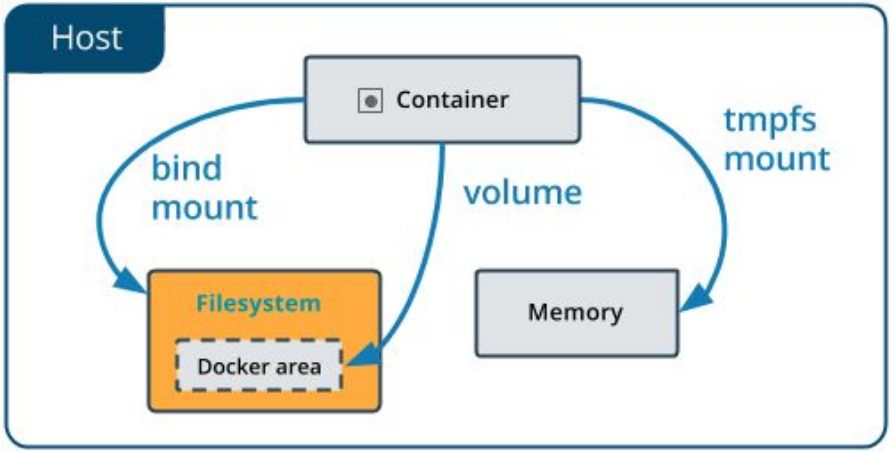
```
WORKDIR '/sim_ws'

ENTRYPOINT ["/bin/bash"]
```

Specifying the `WORKDIR`. If done before `RUN` instructions the default working directory will be the specified one instead of `/`. Also becomes the directory you'll be in when starting an interactive bash session.

Specifying the `ENTRYPOINT`. This is the executable that'll run when starting the container. Note that nothing will happen (for `/bin/bash`) if this is not ran in interactive mode.

https://github.com/f1tenth/f1tenth_gym_ros/blob/ros2_port/Dockerfile

Bind mounts and Volumes



- Bind mount mounts a file or directory on the host machine into a container. The file or directory is referenced by its absolute path on the host machine.
- A volume creates a new directory within Docker's storage directory on the host machine, and Docker manages that directory's contents.

https://docs.docker.com/storage/bind-mounts/

Starting a container with a bind mount

Two ways. `--mount` or `-v` when calling `docker run`.

1. `-v`: Combines all the options together in one field. Consists of three fields, separated by `:`.
   a. First field is the path to the file/directory on the **host machine.**
   b. Second field is the path where the file is mounted in the **container.**
   c. Third field is optional, and is comma-separated list of options.
2. `--mount`: Consists of multiple key-value pairs, separated by commas each consisting of a `<key>=<value>` tuple.
   a. `type`, which can be `bind`, `volume` or `tmpfs`.
   b. `source`, the path to the file/directory on the **host**.
   c. `destination`, the path to the file mounted in the **container**.
   d. Several other options.

https://docs.docker.com/storage/bind-mounts/

Docker network

- Users can create a docker network to connect containers to them, or connect containers to non-Docker workloads.
- Use `$ docker network create <name>` to create a user-defined bridge network.
- Use `$ docker network connect <net-name> <container>` to connect a container to a network.
- Use the `--network host` option when using `docker run` to share the host's network with container.

https://docs.docker.com/network/

Docker Compose

- Compose is a tool for defining and running multiple container at the same time.
- A yaml file `docker-compose.yml` is used to configure.
- Run `docker compose up` or `docker-compose up` (if you've installed the docker-compose binary) to start all your containers.

https://docs.docker.com/compose/

More...

This tutorial is by no means comprehensive.

For documentation: https://docs.docker.com/reference/

Next Time

We'll go over the simulation we'll be using for the course, how to start the Docker containers for the simulation, and how to get started developing your own node in Docker.

Lab 1: Intro to ROS 2

- Will be released today on Canvas
- Due by next tuesday on 23:59, suggested to start as early as possible
- Github link: f1tenth-cmu/Lab-0-Docker-and-ROS-2 (github.com)